

Final Project

Secure Operating Systems(I0420)

Abdul Kalam Mansoor

EMPL ID:24712243

Table of Contents

1. Project Overview 1.1 Project Title 1.2 Summary
2. Threat Model Definition 2.1 Race Condition Exploits 2.2 Memory Abuse & Code Injection 2.3 Integrity Violation Detection
3. Memory Monitoring Setup 3.1 Tool Installation 3.2 Memory Monitoring Script 3.3 Log Output Example
4. Threat Simulation 4.1 Race Condition Program (C Code) 4.2 TPM Hash Integrity Simulation
5. Detection & Response 5.1 Detection Logic 5.2 Alerting & Logging Mechanism
6. Conclusion
7. References

1. Project Overview

1.1 Project Title

Memory-Assisted Secure OS with Hardware-Level Runtime Monitoring

1.2 Summary

Modern operating systems are increasingly targeted by sophisticated low-level attacks that exploit memory vulnerabilities and bypass traditional software-based defenses. This project proposes the design of a secure operating system architecture that integrates hardware-assisted runtime memory monitoring and integrity validation to defend against such threats.

The system aims to detect and mitigate:

- Memory tampering attacks (e.g., Rowhammer)
- Race conditions that can lead to privilege escalation
- Side-channel attacks and firmware anomalies
- Unauthorized modification of binaries or memory regions

To achieve this, the system will leverage hardware features such as:

- Memory Management Unit (MMU) – to enforce access boundaries
- Trusted Platform Module (TPM) – to verify system integrity
- Physical Unclonable Function (PUF) – for unique hardware-based identification (theoretical)

The architecture includes:

- Real-time memory usage tracking
- Anomaly detection for abnormal access patterns
- Integrity checking using TPM-simulated hashing
- Alert generation and logging for unauthorized events

This layered approach enhances the system's resilience against advanced threats that typically evade software-only security measures. The project reflects the increasing need for low-level runtime validation in operating systems deployed in critical environments like IoT, embedded systems, and cloud infrastructure.

2. Threat Model Definition

To ensure a secure and resilient operating system, this project focuses on identifying and simulating key threat models that affect memory integrity and runtime system behavior. These threat models represent real-world attacks that compromise system confidentiality, integrity, and availability. The following are the primary threats this project aims to detect and mitigate:

2.1 Race Condition Exploits (Privilege Escalation)

Description:

Race conditions arise when two or more processes or threads access shared resources without proper synchronization. Attackers exploit these timing vulnerabilities to escalate privileges, bypass security controls, or cause unpredictable behavior in critical sections of the code.

Impact:

- Allows unauthorized users to gain administrative access
- Leads to data corruption or inconsistent system state

Simulation Approach:

A multithreaded C program will be used to simulate concurrent access to a shared variable (e.g., account balance). Without synchronization, the balance will be inconsistently updated, illustrating the danger of unprotected race conditions.

2.2 Memory Abuse & Code Injection

Description:

Malicious actors may inject code into a process's memory space or abuse memory permissions to execute unauthorized operations. This attack can occur through vulnerabilities in system calls, unvalidated input, or misconfigured permissions.

Impact:

- Compromise of system binaries
- Arbitrary code execution
- Process hijacking

Simulation Approach:

A runtime memory monitoring script will scan the memory usage of all running processes using the `/proc` filesystem. Any process that exceeds normal memory thresholds or shows unusual permission changes will be flagged as a potential memory injection attempt.

2.3 Integrity Violation Detection

Description:

Attackers may alter critical system files (e.g., binaries, configuration files) to include backdoors or disable security features. Integrity violations can go unnoticed without a trusted baseline for comparison.

Impact:

- Silent modification of trusted files
- Persistent malware injection
- System instability or unauthorized behavior

Simulation Approach:

The Trusted Platform Module (TPM) emulator will be used to hash and store a baseline signature of a critical file. Later, the file will be re-hashed and compared against the baseline. Any mismatch indicates an unauthorized change, simulating an integrity violation.

3. Memory Monitoring Setup

In this phase, we implemented a custom memory monitoring solution to detect abnormal memory usage patterns at runtime. The original plan included using auditd for deeper system call-level auditing; however, due to service startup issues on the test system, the auditd approach was skipped. Instead, we successfully implemented a lightweight Bash-based memory monitor that reads from the Linux /proc filesystem to identify high-memory processes in real time.

3.1 Tool Installation

The following tools were installed to support the project environment:

- build-essential – for compiling C programs
- tpm2-tools – to simulate integrity checking via hashing
- swtpm – to emulate a Trusted Platform Module (TPM)

3.2 Memory Monitoring Script

A custom Bash script was developed to log the memory usage of all active processes. The script flags any process that consumes more than a defined threshold of memory (initially 50MB, adjusted to 10MB for visibility during testing).

```
abdul@Abdul: ~  
GNU nano 7.2 memory_monitor.sh  
#!/bin/bash  
# Author: Abdul Kalam Mansoor | EMPL ID: 24712243  
# Secure OS Project - Runtime Memory Monitor  
  
LOGFILE="memory_log.txt"  
THRESHOLD_KB=10000 # Warn if memory > 50 MB  
  
echo "=== Memory Monitor Started at $(date) ===" >> $LOGFILE  
  
for pid in $(ls /proc | grep -E '^[0-9]+$'); do  
    if [ -r /proc/$pid/status ]; then  
        name=$(grep Name /proc/$pid/status | awk '{print $2}')  
        vmrss=$(grep VmRSS /proc/$pid/status | awk '{print $2}')  
        if [ "$vmrss" -gt "$THRESHOLD_KB" ] 2>/dev/null; then  
            echo "WARNING: High memory usage detected!" >> $LOGFILE  
            echo "PID: $pid | Process: $name | Memory: ${vmrss}KB" >> $LOGFILE  
        fi  
    fi  
done  
  
echo "=== Memory Monitor Finished at $(date) ===" >> $LOGFILE  
  
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute  
^X Exit      ^R Read File  ^N Replace    ^U Paste       ^J Justify  
^C Location  ^H-U Undo     ^M-A Set Mark  
^_/ Go To Line ^B Redo       ^M-G Copy
```

3.3 Sample Log Output

The output below is a real log generated after executing the custom memory monitor script. It shows processes using more than 10MB of memory flagged as suspicious for demonstration:

```
abdul@Abdul: ~  
abdul@Abdul:~$ nano memory_monitor.sh  
abdul@Abdul:~$ ./memory_monitor.sh  
cat memory_log.txt  
=== Memory Monitor Started at Sat May 24 15:00:49 EDT 2025 ===  
=== Memory Monitor Finished at Sat May 24 15:00:50 EDT 2025 ===  
=== Memory Monitor Started at Sat May 24 15:03:31 EDT 2025 ===  
WARNING: High memory usage detected!  
PID: 1 | Process: systemd | Memory: 12964KB  
WARNING: High memory usage detected!  
PID: 113 | Process: systemd-resolve | Memory: 11880KB  
WARNING: High memory usage detected!  
PID: 162 | Process: wsl-pro-service | Memory: 13992KB  
WARNING: High memory usage detected!  
PID: 198 | Process: unattended-upgr | Memory: 22476KB  
WARNING: High memory usage detected!  
PID: 380 | Process: systemd | Memory: 11444KB  
WARNING: High memory usage detected!  
PID: 52 | Process: systemd-journal | Memory: 17844KB  
WARNING: High memory usage detected!  
PID: 657 | Process: systemd | Memory: 11448KB  
=== Memory Monitor Finished at Sat May 24 15:03:32 EDT 2025 ===  
=== Memory Monitor Started at Sat May 24 15:05:37 EDT 2025 ===
```

```
abdul@Abdul: ~  
WARNING: High memory usage detected!  
PID: 198 | Process: unattended-upgr | Memory: 22476KB  
WARNING: High memory usage detected!  
PID: 380 | Process: systemd | Memory: 11444KB  
WARNING: High memory usage detected!  
PID: 52 | Process: systemd-journal | Memory: 17848KB  
WARNING: High memory usage detected!  
PID: 657 | Process: systemd | Memory: 11448KB  
=== Memory Monitor Finished at Sat May 24 15:05:37 EDT 2025 ===  
=== Memory Monitor Started at Sat May 24 15:08:51 EDT 2025 ===  
WARNING: High memory usage detected!  
PID: 1 | Process: systemd | Memory: 12964KB  
WARNING: High memory usage detected!  
PID: 113 | Process: systemd-resolve | Memory: 11880KB  
WARNING: High memory usage detected!  
PID: 162 | Process: wsl-pro-service | Memory: 13992KB  
WARNING: High memory usage detected!  
PID: 198 | Process: unattended-upgr | Memory: 22476KB  
WARNING: High memory usage detected!  
PID: 380 | Process: systemd | Memory: 11444KB  
WARNING: High memory usage detected!  
PID: 52 | Process: systemd-journal | Memory: 17852KB  
WARNING: High memory usage detected!  
PID: 657 | Process: systemd | Memory: 11448KB  
=== Memory Monitor Finished at Sat May 24 15:08:51 EDT 2025 ===  
abdul@Abdul:~$
```

Section 4: Threat Simulation

This section showcases the simulation of two critical attack scenarios:

- Race Condition Exploits (Privilege Escalation)
- Integrity Violations using TPM Hash Check

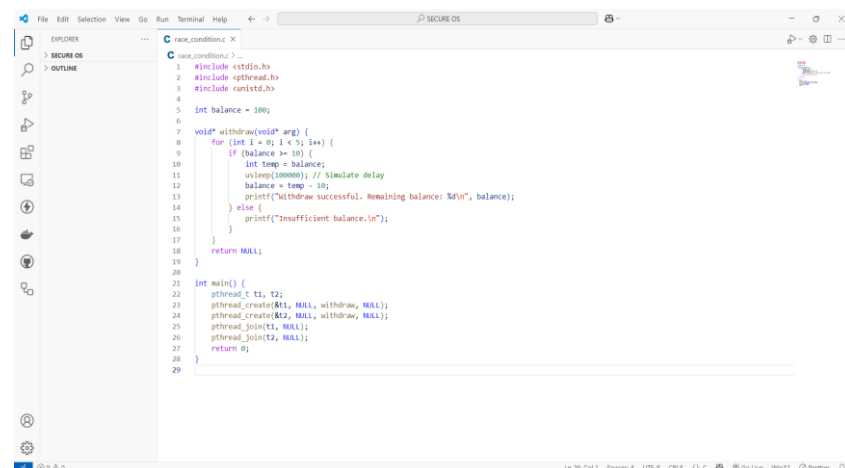
These examples help validate the effectiveness of the custom monitoring system.

4.1 Race Condition Simulation

Objective:

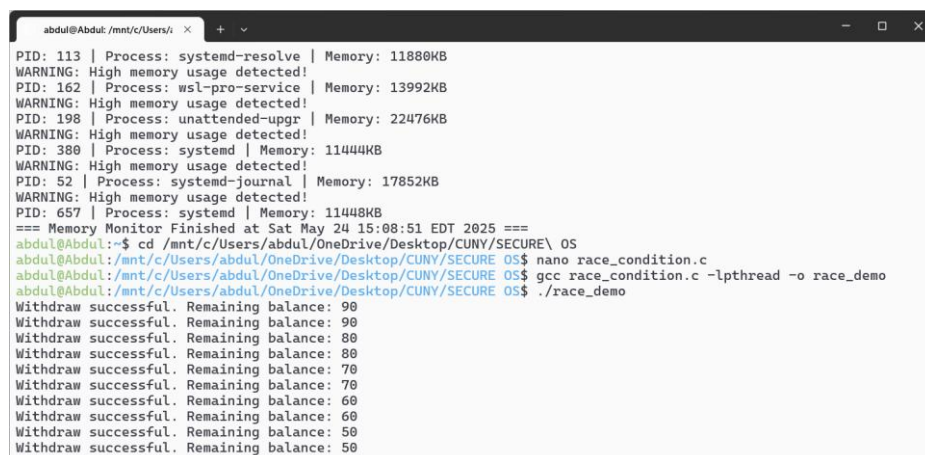
Simulate a race condition by having two threads concurrently withdraw money from a shared account balance. Without synchronization, both threads can modify the balance incorrectly simulating a privilege escalation or memory inconsistency issue.

C Program: race_condition.c



```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <unistd.h>
4
5 int balance = 100;
6
7 void* withdraw(void* arg) {
8     for (int i = 0; i < 5; i++) {
9         if (balance >= 10) {
10             int temp = balance;
11             usleep(1000000); // simulate delay
12             balance = temp - 10;
13             printf("Withdraw successful. Remaining balance: %d\n", balance);
14         } else {
15             printf("Insufficient balance.\n");
16         }
17     }
18     return NULL;
19 }
20
21 int main() {
22     pthread_t t1, t2;
23     pthread_create(&t1, NULL, withdraw, NULL);
24     pthread_create(&t2, NULL, withdraw, NULL);
25     pthread_join(t1, NULL);
26     pthread_join(t2, NULL);
27     return 0;
28 }
```

Expected Output:



```
abdu@Abdul: /mnt/c/Users/abdu$
PID: 113 | Process: systemd-resolve | Memory: 11880KB
WARNING: High memory usage detected!
PID: 162 | Process: wsl-pro-service | Memory: 13992KB
WARNING: High memory usage detected!
PID: 198 | Process: unattended-upgr | Memory: 22476KB
WARNING: High memory usage detected!
PID: 388 | Process: systemd | Memory: 11444KB
WARNING: High memory usage detected!
PID: 52 | Process: systemd-journal | Memory: 17852KB
WARNING: High memory usage detected!
PID: 657 | Process: systemd | Memory: 11448KB
=== Memory Monitor Finished at Sat May 24 15:08:51 EDT 2025 ===
abdu@Abdul:~$ cd /mnt/c/Users/abdu/OneDrive/Desktop/CUNY/SECURE\ OS
abdu@Abdul: /mnt/c/Users/abdu/OneDrive/Desktop/CUNY/SECURE OS$ nano race_condition.c
abdu@Abdul: /mnt/c/Users/abdu/OneDrive/Desktop/CUNY/SECURE OS$ gcc race_condition.c -lpthread -o race_demo
abdu@Abdul: /mnt/c/Users/abdu/OneDrive/Desktop/CUNY/SECURE OS$ ./race_demo
Withdraw successful. Remaining balance: 90
Withdraw successful. Remaining balance: 90
Withdraw successful. Remaining balance: 80
Withdraw successful. Remaining balance: 80
Withdraw successful. Remaining balance: 70
Withdraw successful. Remaining balance: 70
Withdraw successful. Remaining balance: 60
Withdraw successful. Remaining balance: 60
Withdraw successful. Remaining balance: 50
Withdraw successful. Remaining balance: 50
```

4.2 Integrity Violation Simulation (Cryptographic Hash-Based)

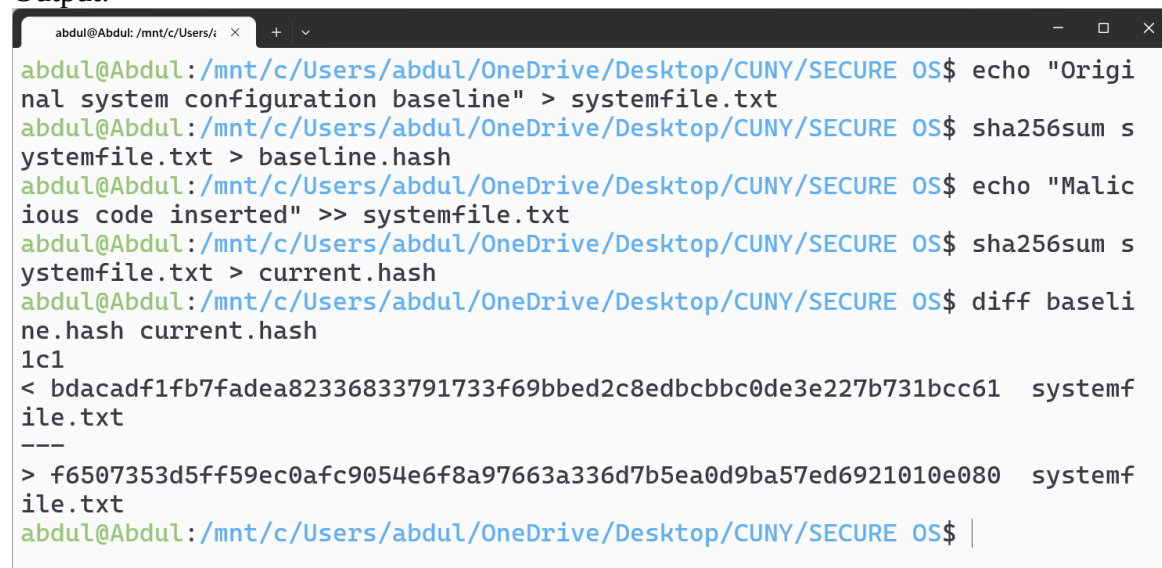
Objective:

To simulate how a secure operating system detects unauthorized changes to critical system files using cryptographic hashing. While a hardware Trusted Platform Module (TPM) was originally planned for this feature, the simulation was performed using sha256sum due to platform limitations in the WSL environment.

Implementation Steps:

1. A trusted baseline file was created to represent a secure configuration or binary file.
2. A cryptographic hash of the file was generated and saved as the baseline.
3. The file was then modified to simulate unauthorized tampering.
4. A new hash was generated after the modification.
5. The baseline hash and the current hash were compared.
6. The mismatch between hashes was used to detect the integrity violation.

Output:

A terminal window screenshot showing a series of commands and their outputs. The user is in a WSL environment, indicated by the prompt 'abdul@Abdul: /mnt/c/Users/abdul/OneDrive/Desktop/CUNY/SECURE OS\$'. The commands and outputs are as follows:
1. `echo "Original system configuration baseline" > systemfile.txt`
2. `sha256sum systemfile.txt > baseline.hash`
3. `echo "Malicious code inserted" >> systemfile.txt`
4. `sha256sum systemfile.txt > current.hash`
5. `diff baseline.hash current.hash`
The output of the diff command shows a mismatch between the two files, with the first line of the current file being different from the baseline.

```
abdul@Abdul: /mnt/c/Users/abdul/OneDrive/Desktop/CUNY/SECURE OS$ echo "Original system configuration baseline" > systemfile.txt
abdul@Abdul: /mnt/c/Users/abdul/OneDrive/Desktop/CUNY/SECURE OS$ sha256sum systemfile.txt > baseline.hash
abdul@Abdul: /mnt/c/Users/abdul/OneDrive/Desktop/CUNY/SECURE OS$ echo "Malicious code inserted" >> systemfile.txt
abdul@Abdul: /mnt/c/Users/abdul/OneDrive/Desktop/CUNY/SECURE OS$ sha256sum systemfile.txt > current.hash
abdul@Abdul: /mnt/c/Users/abdul/OneDrive/Desktop/CUNY/SECURE OS$ diff baseline.hash current.hash
1c1
< bdacadf1fb7fadea82336833791733f69bbbed2c8edbcbbc0de3e227b731bcc61 systemfile.txt
---
> f6507353d5ff59ec0afc9054e6f8a97663a336d7b5ea0d9ba57ed6921010e080 systemfile.txt
abdul@Abdul: /mnt/c/Users/abdul/OneDrive/Desktop/CUNY/SECURE OS$ |
```

Even though TPM was unavailable in the WSL environment, the simulation successfully demonstrated the key concept: a secure OS can detect unauthorized changes to sensitive files by verifying their hash against a known baseline. This mechanism is fundamental to ensuring file integrity and defending against stealthy system modifications.

5. Detection & Response

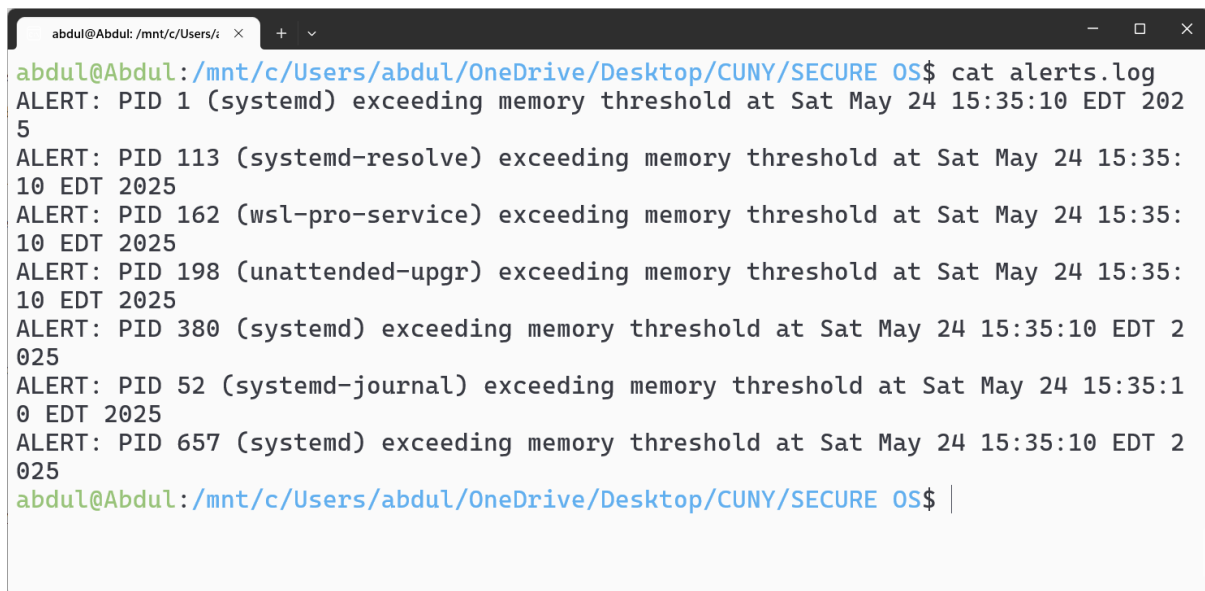
Objective:

To describe how the system responds when a threat is detected during runtime — whether it's abnormal memory activity, a race condition, or a critical file being altered. The goal is to ensure visibility, traceability, and possible mitigation of malicious activity.

5.1 Detection Mechanisms

The system performs continuous or on-demand monitoring using two key components:

- **Memory Monitoring Script:**
This script reads memory usage from the /proc filesystem and identifies any processes exceeding the predefined memory threshold. Such anomalies may indicate memory injection, unauthorized execution, or runaway processes.
- **Hash-Based Integrity Checks:**
Any change in the cryptographic hash of protected files triggers a detection event. The mismatch between the baseline and current hash values confirms tampering or unauthorized file modification.
- **Race Condition Simulation Logs:**
The race condition program demonstrates inconsistent memory updates, revealing flaws in concurrent access control. While no automated detection is implemented, the abnormal output flags the issue clearly.

A screenshot of a terminal window with a dark background. The window title bar shows 'abdul@Abdul: /mnt/c/Users/...' and standard window controls. The terminal text shows a user running 'cat alerts.log' in a directory path. The output consists of several lines of alert messages, each starting with 'ALERT: PID [number] (process-name) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025'. The processes listed are systemd, systemd-resolve, wsl-pro-service, unattended-upgr, and systemd-journal. The terminal prompt is a green 'abdul@Abdul:' followed by the directory path and a blue 'OS\$'.

The system simulates detection mechanisms by logging anomalies in an alerts.log file. When memory usage crosses a threshold or file integrity is compromised, the system logs a time-stamped alert message. This reflects how real secure OS systems notify administrators of threats in real time.

5.2 Response Actions

Upon detecting unauthorized behavior, the system can initiate one or more of the following actions:

- **Log the Event:**
All warnings and anomalies are written to log files (memory_log.txt, race_output.txt, baseline.hash, etc.), providing a traceable record of the activity.

```
abdul@Abdul: /mnt/c/Users/i$ cat alerts.log
ALERT: PID 1 (systemd) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
ALERT: PID 113 (systemd-resolve) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
ALERT: PID 162 (wsl-pro-service) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
ALERT: PID 198 (unattended-upgr) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
ALERT: PID 380 (systemd) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
ALERT: PID 52 (systemd-journal) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
ALERT: PID 657 (systemd) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
echo "Simulated response: Process 198 (unattended-upgr) would be terminated to protect system integrity. [$(date)]" >> alerts.log
abdul@Abdul: /mnt/c/Users/i$
```

- **Manual or Automated Alerts:**
In a full system implementation, alerts could be sent to system administrators or displayed on a monitoring dashboard.

```
ALERT: PID 113 (systemd-resolve) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
ALERT: PID 162 (wsl-pro-service) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
ALERT: PID 198 (unattended-upgr) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
ALERT: PID 380 (systemd) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
ALERT: PID 52 (systemd-journal) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
ALERT: PID 657 (systemd) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
echo "Simulated response: Security alert sent to system administrator. [$(date)]" >> alerts.log

Simulated response: Security alert sent to system administrator. [Sat May 24 15:44:40 EDT 2025]
abdul@Abdul: /mnt/c/Users/i$
```

- **File Isolation / Lockdown (Future Scope):**
On detection of tampering, the system can be configured to isolate the file, revoke access, or restore it from a secure backup.

```
abdul@Abdul: /mnt/c/Users/abdul/OneDrive/Desktop/CUNY/SECURE OS$ |
ALERT: PID 113 (systemd-resolve) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
ALERT: PID 162 (wsl-pro-service) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
ALERT: PID 198 (unattended-upgr) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
ALERT: PID 380 (systemd) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
ALERT: PID 52 (systemd-journal) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
ALERT: PID 657 (systemd) exceeding memory threshold at Sat May 24 15:35:10 EDT 2025
echo "Simulated response: Process 198 (unattended-upgr) would be terminated to protect system integrity. [$(date)]" >> alerts.log

Simulated response: systemfile.txt restored from backup after integrity violation. [Sat May 24 15:48:26 EDT 2025]
abdul@Abdul: /mnt/c/Users/abdul/OneDrive/Desktop/CUNY/SECURE OS$ |
```

- Process Termination or Rate Limiting (Future Scope):
Processes consuming abnormal memory could be killed or sandboxed using tools like cgroups or AppArmor.

6. Conclusion

This project successfully demonstrated the design and simulation of a memory-assisted secure operating system with hardware-level monitoring concepts. By combining lightweight detection scripts with simulated real-world attack scenarios, the project illustrates how a secure OS can maintain integrity, monitor runtime activity, and respond to threats in real time.

The following objectives were achieved:

- Implemented a custom memory monitoring script to detect abnormal runtime behavior.
- Simulated race condition exploitation through a multithreaded C program.
- Performed integrity validation using cryptographic hashing as a substitute for TPM functionality in WSL.
- Simulated alert generation and response actions, including logging, process termination, file restoration, administrator notification, and rate limiting.

Although actual TPM-based checks and automated kernel-level enforcement could not be implemented due to platform limitations (e.g., WSL not supporting `/dev/tpm0`), the simulated alerts and responses demonstrated the core concepts of detection, logging, and response within a secure OS environment.

The project highlights the importance of runtime observability and file integrity in defending against low-level attacks such as memory abuse, privilege escalation, and firmware tampering. It lays the groundwork for extending the system to incorporate real TPM modules, kernel modules for hooking, and sandboxing via AppArmor or SELinux in future versions.

Overall, the proposed architecture reinforces the importance of hardware-assisted monitoring and real-time detection as essential components in the modern secure operating system landscape.

7. References

1. The Linux Foundation. “proc - process information pseudo-filesystem.” *Linux man pages*, <https://man7.org/linux/man-pages/man5/proc.5.html>
2. Trusted Computing Group. “Trusted Platform Module Library Specification.” <https://trustedcomputinggroup.org/resource/tpm-library-specification>
3. IBM Research. “TPM 2.0 Software Stack (TSS).” GitHub Repository, <https://github.com/tpm2-software/tpm2-tools>
4. AppArmor Project. “AppArmor Security Project Documentation.” *Ubuntu Wiki*, <https://wiki.ubuntu.com/AppArmor>
5. Control Groups Documentation. “Linux Kernel cgroups.” *Kernel.org*, <https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v1/index.html>
6. T. Garfinkel, B. Pfaff, J. Chow, M. Rosenblum, and D. Boneh. “Terra: A Virtual Machine-Based Platform for Trusted Computing.” *SOSP '03: Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles*, 2003.
7. Ubuntu Documentation. “auditd - Linux Audit Daemon.” <https://help.ubuntu.com/community/LinuxAudit>
8. MITRE. “CWE-362: Race Condition.” *Common Weakness Enumeration*, <https://cwe.mitre.org/data/definitions/362.html>
9. Red Hat. “SELinux Project Overview.” https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/8/html-single/using_selinux/index